Hashing

Md. Shahriar Kabir

January 2026

1 Hashing

1.1 Introduction

Hashing is a technique used to efficiently store and retrieve data by mapping keys to positions in a fixed-size table using a hash function.

The main goal of hashing is to achieve:

$$\text{Average Time Complexity} = O(1)$$

for insertion, deletion, and searching.

1.2 Hash Table

A **hash table** is a data structure that stores key-value pairs. Each key is mapped to an index using a hash function.

1.3 Hash Function

A hash function maps a key k to an index in the hash table.

$$h(k) = k \bmod m$$

where:

- k = key
- m = size of hash table

1.4 Collision

A **collision** occurs when two different keys are mapped to the same hash table index.

$$h(k_1) = h(k_2), \quad k_1 \neq k_2$$

1.5 Collision Resolution Techniques

- Separate Chaining
- Open Addressing
  - Linear Probing
  - Quadratic Probing
  - Double Hashing

2 Linear Probing

2.1 Definition

Linear probing is an open addressing collision resolution technique where, upon collision, the algorithm checks the next available slot sequentially until an empty position is found.

2.2 Probe Sequence

If a collision occurs at index $h(k)$, the probe sequence is:

$$(h(k) + i) \bmod m, \quad i = 0, 1, 2, \ldots$$

2.3 Insertion Algorithm

```
LINEAR_PROBING_INSERT(T, k, m):
    i ← 0
    while i < m:
        index ← (h(k) + i) mod m
        if T[index] is EMPTY:
            T[index] ← k
            return
        i ← i + 1
    report "Hash Table Overflow"
```

2.4 Search Algorithm

```
LINEAR_PROBING_SEARCH(T, k, m):
    i ← 0
    while i < m:
        index ← (h(k) + i) mod m
        if T[index] == k:
            return FOUND
        if T[index] is EMPTY:
            return NOT FOUND
```

```
        i ← i + 1
    return NOT FOUND
```

2.5 Deletion Algorithm

In linear probing, deletion cannot simply mark a slot as empty. Instead, a special marker called **DELETED** is used.

```
LINEAR_PROBING_DELETE(T, k, m):
    i ← 0
    while i < m:
        index ← (h(k) + i) mod m
        if T[index] == k:
            T[index] ← DELETED
            return
        if T[index] is EMPTY:
            return
        i ← i + 1
```

3 Worked Example

3.1 Given

- Hash table size: $m = 10$
- Hash function: $h(k) = k \bmod 10$
- Keys to insert: $23, 43, 13, 27, 33, 17$

3.2 Step-by-Step Insertion

Insert 23:

$$h(23) = 3$$

Insert 43:

$$h(43) = 3 \rightarrow \text{collision}$$

Probe sequence: $3 \rightarrow 4$

Insert 13:

$$h(13) = 3 \rightarrow 3 \rightarrow 4 \rightarrow 5$$

Insert 27:

$$h(27) = 7$$

Insert 33:

$$h(33) = 3 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6$$

Insert 17:

$$h(17) = 7 \rightarrow 8$$

3.3 Final Hash Table

Index	0	1	2	3	4	5	6	7	8
9									
Key	–	–	–	23	43	13	33	27	17
–									

3.4 Probe Sequence Summary

Key	h(k)	Probe Sequence	Final Index
23	3	3	3
43	3	$3 \to 4$	4
13	3	$3 \to 4 \to 5$	5
27	7	7	7
33	3	$3 \to 4 \to 5 \to 6$	6
17	7	$7 \to 8$	8

4 Load Factor

The load factor α is defined as:

$$\alpha = \frac{n}{m}$$

For this example:

$$\alpha = \frac{6}{10} = 0.6$$

5 Performance Analysis

- Average successful search:

$$O\left(\frac{1}{2}\left(1 + \frac{1}{1-\alpha}\right)\right)$$

- Average unsuccessful search:

$$O\left(\frac{1}{1-\alpha}\right)$$

- Worst case:

$$O(n)$$

6 Primary Clustering

6.1 Definition

Primary clustering occurs when a contiguous block of occupied slots forms, increasing the length of probe sequences for new insertions.

6.2 Cause

- Sequential probing
- Repeated collisions

7 Advantages

- Simple to implement
- No extra memory required
- Cache-friendly

8 Disadvantages

- Suffers from primary clustering
- Performance degrades as load factor increases
- Deletion is complex

9 Comparison with Chaining

Feature	Linear Probing	Chaining
Extra Memory	No	Yes
Clustering	Yes	No
Load Factor	< 1	Can exceed 1
Cache Friendly	Yes	No

10 Common Exam Mistakes

- Forgetting modulo wrap-around
- Marking deleted slots as empty
- Skipping probe steps
- Incorrect load factor calculation

11 Important Exam Questions

Mid-Level

- Explain linear probing with an example.
- Define primary clustering.
- Why is deletion difficult in linear probing?

Hard

- Analyze the expected search time of linear probing.
- Compare linear probing with quadratic probing.
- Explain the effect of load factor on performance.

12 One-Line Summary

Linear probing is an open addressing collision resolution technique where collisions are resolved by sequentially probing the next available slots in the hash table.

13 Quadratic Probing

13.1 Introduction

Quadratic Probing is a collision resolution technique used in hashing under the category of **open addressing**. It improves upon linear probing by reducing the problem of primary clustering.

13.2 Definition

Quadratic Probing resolves collisions by probing hash table indices using a quadratic function of the probe number.

Instead of checking consecutive slots, it checks slots at increasing quadratic distances.

13.3 Hash Function

The initial hash function is:

$$h(k) = k \bmod m$$

where:

- k = key
- m = size of hash table

13.4 Probe Sequence

If a collision occurs at index $h(k)$, the probe sequence in quadratic probing is:

$$h(k, i) = (h(k) + c_1 i + c_2 i^2) \bmod m$$

In most exam problems:

$$h(k, i) = (h(k) + i^2) \bmod m$$

where $i = 0, 1, 2, \ldots$

13.5 Why Quadratic Probing?

Linear probing suffers from **primary clustering**, where long contiguous blocks of occupied slots form. Quadratic probing spreads out probes, thereby reducing primary clustering.

14 Insertion Algorithm

```
QUADRATIC_PROBING_INSERT(T, k, m):
    i ← 0
    while i < m:
        index ← (h(k) + i*i) mod m
        if T[index] is EMPTY:
            T[index] ← k
            return
        i ← i + 1
    report "Hash Table Overflow"
```

15 Search Algorithm

```
QUADRATIC_PROBING_SEARCH(T, k, m):
    i ← 0
    while i < m:
        index ← (h(k) + i*i) mod m
        if T[index] == k:
            return FOUND
        if T[index] is EMPTY:
            return NOT FOUND
        i ← i + 1
    return NOT FOUND
```

16 Deletion Algorithm

Deletion in quadratic probing uses a **DELETED** marker to avoid breaking probe sequences.

```
QUADRATIC_PROBING_DELETE(T, k, m):
    i ← 0
    while i < m:
```

```
index ← (h(k) + i*i) mod m
if T[index] == k:
    T[index] ← DELETED
    return
if T[index] is EMPTY:
    return
i ← i + 1
```

17 Worked Example

17.1 Given

- Hash table size: $m = 11$
- Hash function: $h(k) = k \bmod 11$
- Collision resolution: Quadratic Probing
- Keys to insert: $21, 32, 43, 54, 65$

17.2 Step-by-Step Insertion

Insert 21:

$$h(21) = 10$$

Insert 32:

$$h(32) = 10 \rightarrow \text{collision}$$
$$(10 + 1^2) \bmod 11 = 0$$

Insert 43:

$$h(43) = 10 \rightarrow 10 \rightarrow 0$$
$$(10 + 2^2) \bmod 11 = 3$$

Insert 54:

$$h(54) = 10 \rightarrow 10 \rightarrow 0 \rightarrow 3$$
$$(10 + 3^2) \bmod 11 = 8$$

Insert 65:

$$h(65) = 10 \rightarrow 10 \rightarrow 0 \rightarrow 3 \rightarrow 8$$
$$(10 + 4^2) \bmod 11 = 4$$

17.3 Final Hash Table

Index	Key
0	32
3	43
4	65
8	54
10	21

18 Probe Sequence Summary

Key	Probe Sequence	Final Index
21	10	10
32	$10 \to 0$	0
43	$10 \to 0 \to 3$	3
54	$10 \to 0 \to 3 \to 8$	8
65	$10 \to 0 \to 3 \to 8 \to 4$	4

19 Secondary Clustering

19.1 Definition

Secondary clustering occurs when keys that hash to the same initial index follow the same probe sequence.

19.2 Comparison with Linear Probing

- Linear Probing $\to$ Primary clustering
- Quadratic Probing $\to$ Secondary clustering

20 Performance Analysis

Let load factor:

$$\alpha = \frac{n}{m}$$

- Average Case: $O(1)$
- Worst Case: $O(n)$
- Efficient when $\alpha < 0.5$

21 Conditions for Successful Quadratic Probing

- Table size m should be a prime number
- Load factor should be less than 0.5

22 Advantages

- Reduces primary clustering
- Better distribution than linear probing
- Simple to implement

23 Disadvantages

- Suffers from secondary clustering
- May not visit all table slots
- More complex than linear probing

24 Comparison with Linear Probing

Feature	Linear Probing	Quadratic Probing
Probe Type	Linear	Quadratic
Primary Clustering	Yes	No
Secondary Clustering	No	Yes
Implementation	Simple	Moderate

24.1 Theorem and Proof

Theorem: If the hash table size m is a prime number and the load factor $\alpha \leq 0.5$, then quadratic probing will always find an empty slot if one exists.

—

Quadratic Probing Function

The quadratic probing sequence is defined as:

$$h(k, i) = (h(k) + i^2) \bmod m, \quad i = 0, 1, 2, \ldots$$

—

Given

- Hash table size m is prime
- Load factor $\alpha = \frac{n}{m} \leq 0.5$
- Number of stored keys $n \leq \frac{m}{2}$

Thus, at least $\frac{m}{2}$ slots are empty.

—

Proof

Consider the probe sequence:

$$h(k, 0), h(k, 1), h(k, 2), \ldots, h(k, t)$$

Assume, for contradiction, that quadratic probing fails to find an empty slot even though one exists. Then all probed positions must be occupied.

—

Step 1: Distinctness of Probe Positions For a prime table size m, the values:

$$i^2 \bmod m \quad \text{for } i = 0, 1, 2, \ldots, \frac{m-1}{2}$$

are all distinct.

Reason: If $i^2 \equiv j^2 \pmod{m}$, then:

$$(i-j)(i+j) \equiv 0 \pmod{m}$$

Since m is prime, this implies:

$$i \equiv j \pmod{m} \quad \text{or} \quad i \equiv -j \pmod{m}$$

Within the range $0 \leq i, j \leq \frac{m-1}{2}$, this implies $i = j$.

Thus, the first $\frac{m+1}{2}$ probes visit **distinct** table positions.

—

Step 2: Number of Probes vs Occupied Slots Quadratic probing examines:

$$\frac{m+1}{2} \text{ distinct slots}$$

But the number of occupied slots is:

$$n \leq \frac{m}{2}$$

Hence, among the probed positions:

$$\text{number of occupied slots} < \text{number of probes}$$

By the pigeonhole principle, at least one probed slot must be empty.

—

Step 3: Contradiction This contradicts the assumption that quadratic probing fails to find an empty slot when one exists.

—

Conclusion

Therefore, if:

- the table size m is prime, and
- the load factor $\alpha \leq 0.5$

quadratic probing is guaranteed to find an empty slot if one exists.

Hence proved.

25 Common Exam Mistakes

- Forgetting square term in probing
- Using non-prime table size
- Assuming quadratic probing eliminates all clustering
- Ignoring load factor condition

26 Important Exam Questions

Mid-Level

- Explain quadratic probing with an example.
- What is secondary clustering?
- Why quadratic probing is better than linear probing?

Hard

- Prove that quadratic probing does not guarantee visiting all slots.
- Compare linear probing, quadratic probing, and double hashing.
- Analyze the performance of quadratic probing using load factor.

27 One-Line Summary

Quadratic probing is an open addressing collision resolution technique that reduces primary clustering by using quadratic increments in the probe sequence.

28 Double Hashing

28.1 Introduction

Double Hashing is an open addressing collision resolution technique used in hashing. When a collision occurs, a second hash function is used to determine the probe step size. This reduces clustering problems present in linear and quadratic probing.

—

28.2 Hash Functions

Let:

$$h_1(k) = k \bmod m$$

$$h_2(k) = R - (k \bmod R)$$

where:

- m = size of hash table
- R = a prime number smaller than m
- $h_2(k) \neq 0$

—

28.3 Probe Sequence

The probe sequence for key k is:

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$$

where $i = 0, 1, 2, \ldots$

—

28.4 Algorithm (Insertion)

```
Compute h1(k)
if slot is empty then
  Insert key
else
  Compute h2(k)
  for i = 1 to m − 1 do
    index = (h1(k) + i · h2(k)) mod m
    if slot is empty then
      Insert key
      break
    end if
  end for
end if
```

—

28.5 Worked Example

Given:

$$m = 11, \quad R = 7$$

$$\text{Keys: } 22, 1, 13, 11, 24, 33$$

$$h_1(k) = k \bmod 11$$
$$h_2(k) = 7 - (k \bmod 7)$$

—

Step-by-Step Insertion

Key 22:

$$h_1(22) = 0$$

Insert at index 0.

Key 1:

$$h_1(1) = 1$$

Insert at index 1.

Key 13:

$$h_1(13) = 2$$

Insert at index 2.

Key 11:

$$h_1(11) = 0 \quad \text{(collision)}$$
$$h_2(11) = 7 - (11 \bmod 7) = 3$$
$$(0 + 1 \cdot 3) \bmod 11 = 3$$

Insert at index 3.

Key 24:

$$h_1(24) = 2 \quad \text{(collision)}$$
$$h_2(24) = 7 - (24 \bmod 7) = 4$$
$$(2 + 1 \cdot 4) \bmod 11 = 6$$

Insert at index 6.

Key 33:

$$h_1(33) = 0 \quad \text{(collision)}$$
$$h_2(33) = 7 - (33 \bmod 7) = 2$$
$$(0 + 1 \cdot 2) \bmod 11 = 2 \quad \text{(occupied)}$$
$$(0 + 2 \cdot 2) \bmod 11 = 4$$

Insert at index 4.

—

28.6 Final Hash Table

Index	Key
0	22
1	1
2	13
3	11
4	33
5	—
6	24
7	—
8	—
9	—
10	—

—

28.7 Advantages

- Eliminates primary clustering
- Reduces secondary clustering
- Uniform distribution of probes

—

28.8 Disadvantages

- Requires two hash functions
- More computation overhead
- Poor choice of $h_2(k)$ can fail probing

—

28.9 Time Complexity

Average case: $O(1)$

Worst case: $O(n)$

—

28.10 Comparison with Other Methods

Method	Clustering	Probing Pattern	Efficiency
Linear Probing	Primary	Sequential	Low
Quadratic Probing	Secondary	Polynomial	Medium
Double Hashing	None	Randomized	High

—

28.11 Important Exam Notes

- $h_2(k)$ must be relatively prime to table size
- Table size should be prime
- Load factor must be < 0.7

—

28.12 Typical Exam Questions

1. Explain double hashing with an example.
2. Compare linear, quadratic, and double hashing.
3. Why must $h_2(k)$ be non-zero?
4. Simulate double hashing for given keys.

28.13 Comparison of Linear Probing, Quadratic Probing, and Double Hashing

Feature	**Linear Probing**	**Quadratic Probing**	**Double Hashing**
Probe Formula	$h(k,i) = (h(k)+i) \bmod m$	$h(k,i) = (h(k) + i^2) \bmod m$	$h(k,i) = (h_1(k) + i \cdot h_2(k)) \bmod m$
Clustering Type	Primary clustering occurs	Secondary clustering occurs	No primary or secondary clustering
Collision Resolution	Sequential probing	Polynomial spacing	Variable step size using second hash
Distribution of Keys	Poor	Moderate	Good (near-uniform)
Number of Hash Functions	One	One	Two
Efficiency	Low	Medium	High
Implementation Complexity	Simple	Moderate	Complex
Performance at High Load Factor	Poor	Better than linear	Best among the three

Conclusion: Double hashing provides the best performance by minimizing clustering, whereas linear probing suffers from primary clustering and quadratic probing from secondary clustering.

—

28.14 Why Must $h_2(k)$ Be Non-Zero?

In double hashing, the probe sequence is given by:

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$$

If $h_2(k) = 0$:

$$h(k, i) = h_1(k) \quad \forall i$$

This causes the algorithm to repeatedly probe the same index, leading to:

- Infinite loop
- Failure to resolve collisions
- Inability to insert the key

Therefore:

- $h_2(k)$ must be non-zero
- $h_2(k)$ should be relatively prime to table size m
- This ensures all table slots can be probed

Exam Line: To guarantee successful probing and avoid infinite collisions, the second hash function must generate a non-zero step size.

28.15 Proof

For an open-address hash table with load factor

$$\alpha = \frac{n}{m} < 1,$$

assuming uniform hashing, the expected number of probes in an unsuccessful search is at most

$$\frac{1}{1-\alpha}.$$

Let the hash table have m slots, of which n are occupied and $m - n$ are empty. The load factor is

$$\alpha = \frac{n}{m}.$$

In an unsuccessful search using open addressing, probes continue until an empty slot is found. Under the assumption of uniform hashing, each probe is equally likely to examine any slot in the table.

The probability that a randomly probed slot is empty is

$$P(\text{empty}) = \frac{m-n}{m} = 1 - \alpha.$$

Thus, an unsuccessful search can be modeled as a geometric random variable X, where each probe is a trial and success corresponds to finding an empty slot, with success probability $p = 1 - \alpha$.

The expected value of a geometric random variable is

$$E[X] = \frac{1}{p}.$$

Substituting $p = 1 - \alpha$, we obtain

$$E[X] = \frac{1}{1 - \alpha}.$$

Hence, the expected number of probes in an unsuccessful search is at most

$$\frac{1}{1 - \alpha}.$$